MACHINE LEARNING

# Learning a Model: Decision Trees

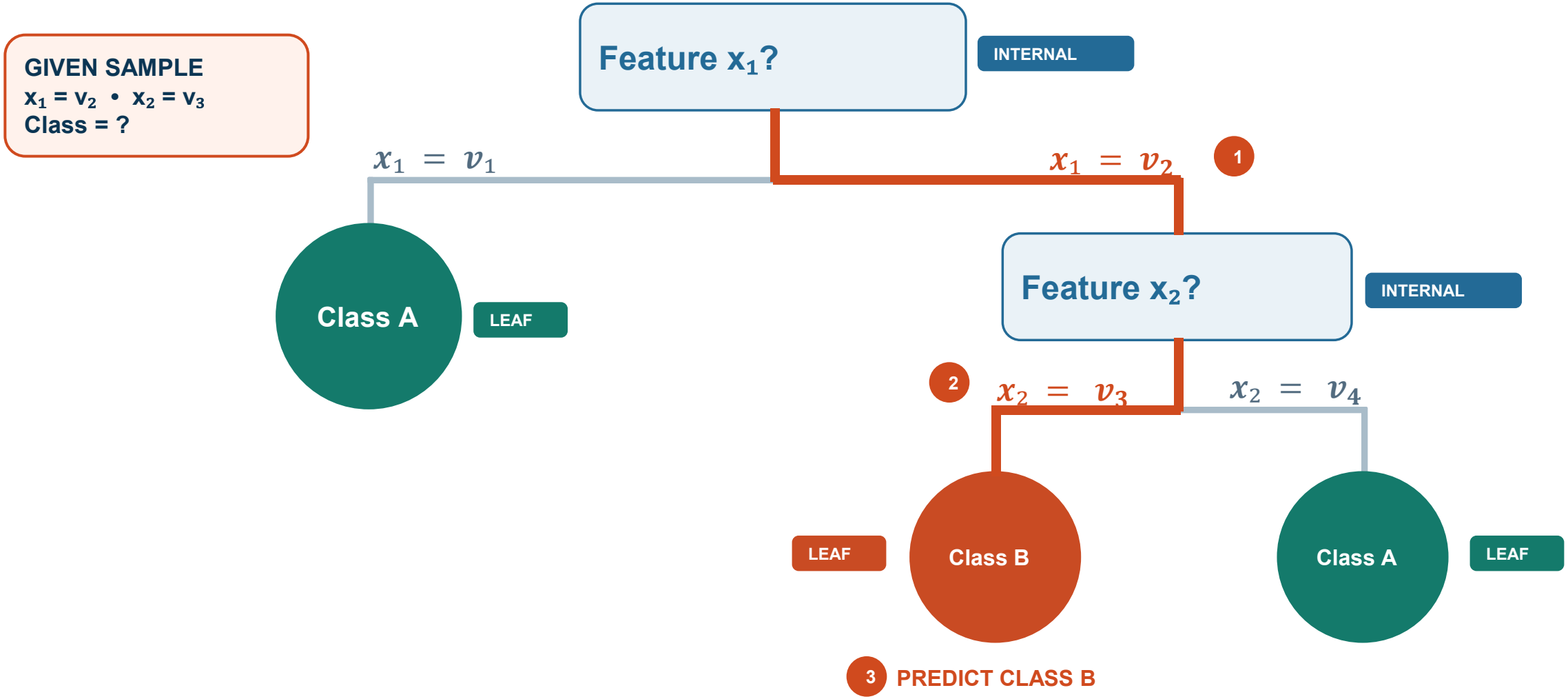
## Decision Trees

# Trees and nearest neighbors

| | |
|---|---|
| **Nearest neighbors** | Compare the new input with retained labeled samples. Work occurs at prediction time. |
| **Decision tree** | Follow the learned feature questions to a stored leaf prediction. |
| **Where the work occurs** | Tree fitting searches for the best tree. Prediction applies the resulting rule. |

# A sample follows feature-value branches to a class prediction

For $x_1 = v_2$ and $x_2 = v_3$, the tree routes the sample to Class B.



# Accuracy and tree size

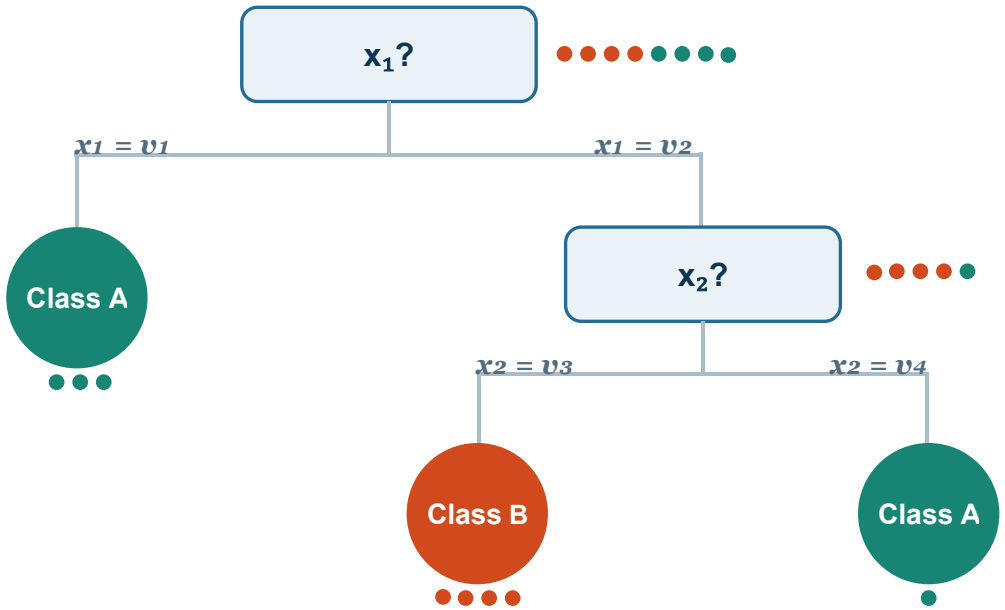
| | |
|---|---|
| **Predictive aim** | Predict new samples accurately. |
| **Complexity preference** | If accuracy is similar, prefer fewer questions. |

**Why are smaller trees preferred?**

# The tree-learning algorithm

At each node:

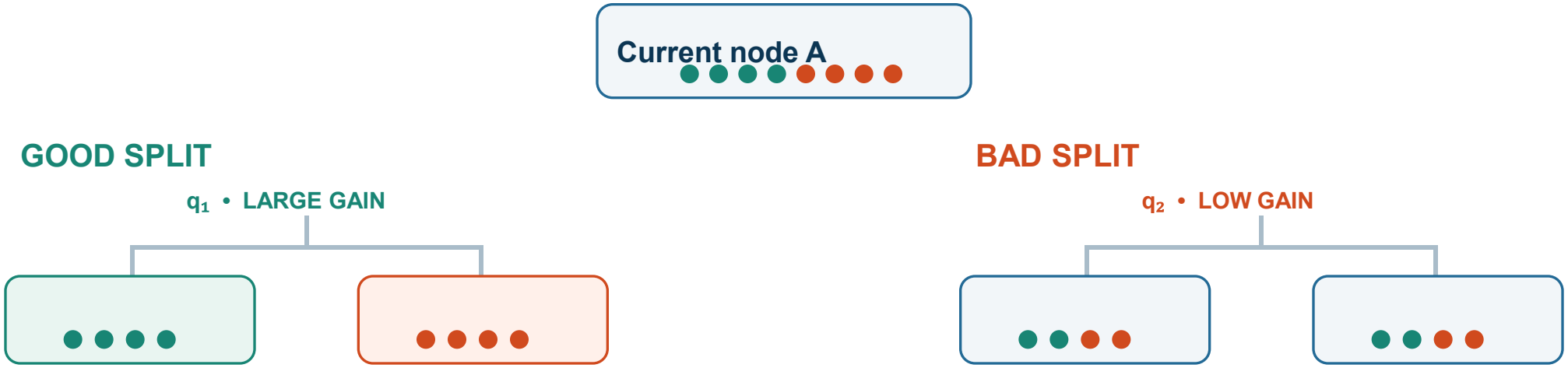
1. If the node is pure, return its class.
2. If no valid split remains, return the majority class.
3. Select the best feature to ask about at this node.
4. Split samples based on the answer and send them to child nodes.
5. Run the recursive algorithm for each child node.




Samples are repartitioned at each question; leaves return a class.

# Good questions separate classes; bad questions preserve mixtures

Compare candidate splits by the size-weighted impurity of their children.




$$\Delta\text{Impurity}(q;\ A)\ =\ \text{Impurity}(A)\ -\ \sum_{j}(n_j/n)\ \cdot\ \text{Impurity}(A_j)$$

# The tree-learning algorithm

At each node:

1. If the node is pure, return its class.
2. If no valid split remains, return the majority class.
3. Score every valid question using ΔImpurity.
4. Choose the largest gain. Stop if the gain is too small.
5. Split the node and repeat in each child.

# Information gain and Gini gain use the same weighted pattern

For a candidate question q, compare the parent impurity with the size-weighted impurity of its child sets $A_i$.

## Entropy / information gain

$$H(A) = -\sum_{k} p_k \log_2 p_k$$

$$\boldsymbol{IG(A,q) = H(A) - \sum_{i} (|A_i| / |A|)\, H(A_i)}$$

Unit: bits when $\log_2$ is used

**Choose q with maximum IG.**

## Gini / impurity reduction

$$G(A) = 1 - \sum_{k} {p_k}^2$$

$$\boldsymbol{GR(A,q) = G(A) - \sum_{i} (|A_i| / |A|)\, G(A_i)}$$

Unitless; binary range is 0 to 0.5

**Choose q with maximum Gini reduction.**

**Same structure: parent impurity − weighted child impurity**

# Spam detection turns email attributes into a classification task

Samples 1–8

| ID | Thread | Phrase | Link | Att. | Known | Class |
|---|---|---|---|---|---|---|
| 1 | yes | no | no | no | yes | Not spam |
| 2 | yes | no | yes | no | yes | Not spam |
| 3 | yes | no | no | yes | yes | Not spam |
| 4 | yes | no | yes | yes | no | Not spam |
| 5 | yes | no | no | no | no | Not spam |
| 6 | yes | no | yes | no | yes | Not spam |
| 7 | yes | yes | yes | yes | no | Spam |
| 8 | no | no | yes | yes | no | Spam |

Samples 9–16

| ID | Thread | Phrase | Link | Att. | Known | Class |
|---|---|---|---|---|---|---|
| 9 | no | no | yes | yes | yes | Spam |
| 10 | no | yes | yes | yes | no | Spam |
| 11 | no | no | yes | yes | yes | Spam |
| 12 | no | no | yes | no | yes | Not spam |
| 13 | no | no | no | no | yes | Not spam |
| 14 | no | yes | no | no | yes | Not spam |
| 15 | no | no | no | yes | yes | Not spam |
| 16 | no | no | no | no | no | Spam |

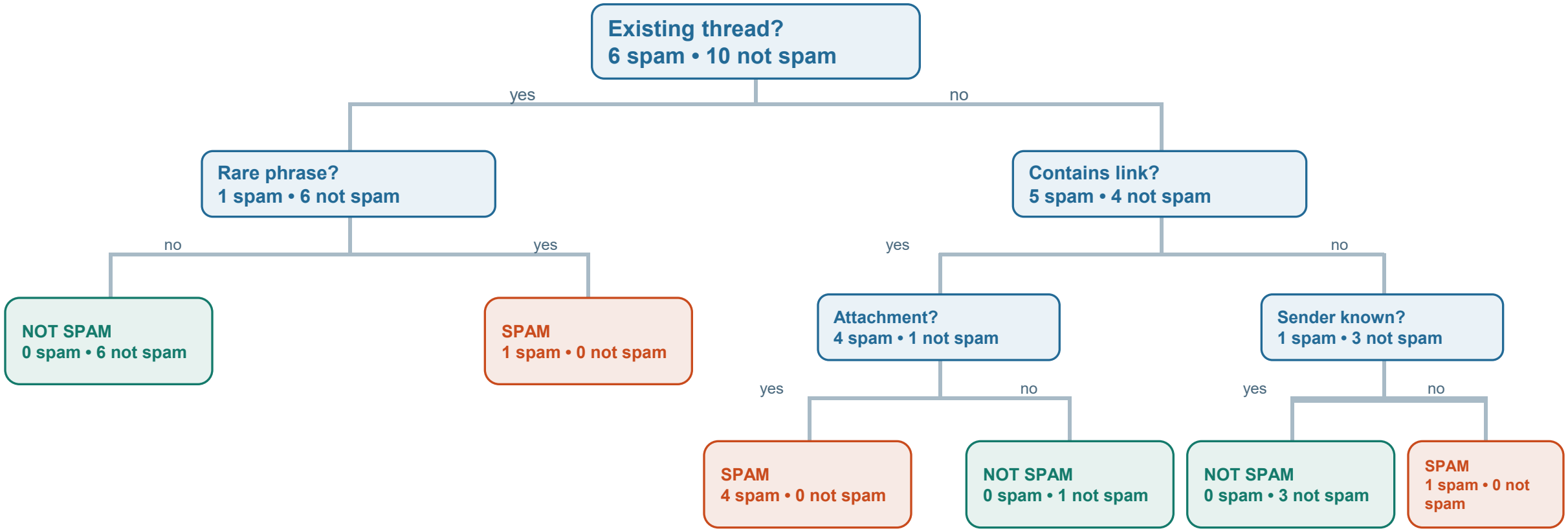
## Example definition

**INPUT**
**Five binary email attributes:**
- **Existing email thread?**
- **Contains a rare email phrase?**
- **Has a link?**
- **Has attachments?**
- **Is the sender known?**

**TARGET**
- **Spam vs. Not spam**

# A fully grown tree can predict every training exception

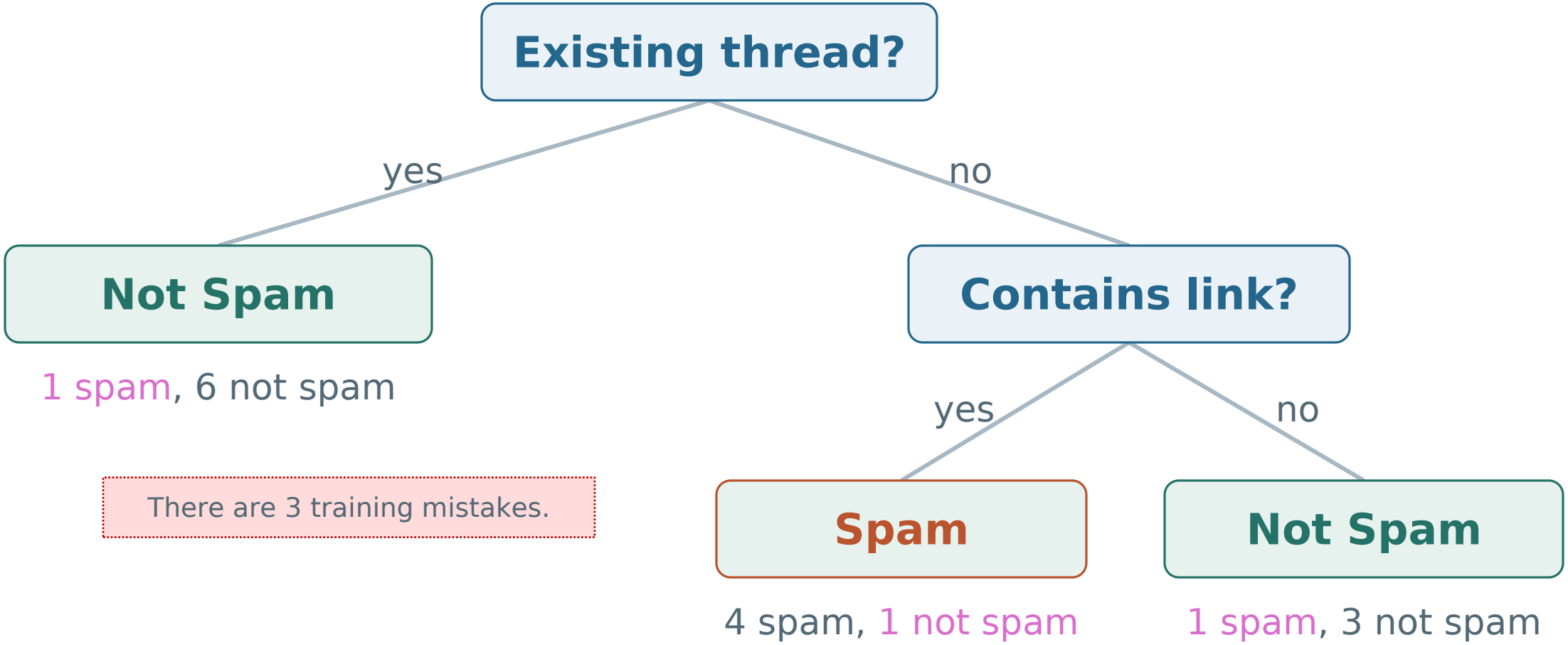
Extra questions isolate three one-sample exceptions and drive training error to zero.




**Training error = 0 • Three leaves contain only one sample**

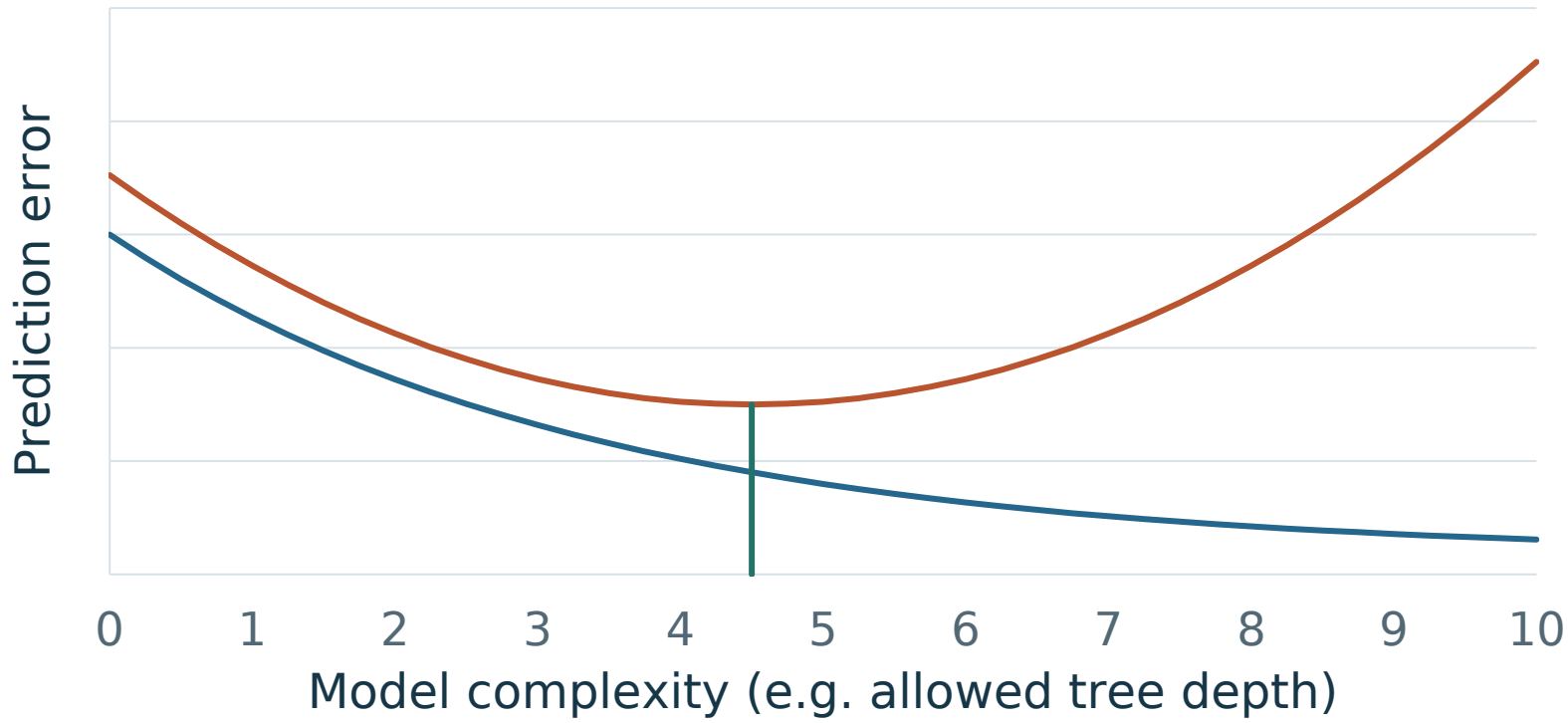
The extra questions fit exceptions rather than a stable pattern.

# Or we can stop here



# Model complexity and generalization

Compare errors as model flexibility increases, using the same training and validation sets.




Schematic curves. Actual validation curves may be irregular.

**Underfitting**

Both errors are high.

**Useful complexity**

Validation error is lowest.

**Overfitting**

Training error falls while validation error rises.

# Bias is systematic error in the average prediction

Fix an input x, then retrain the same tree-building method on many possible datasets D.

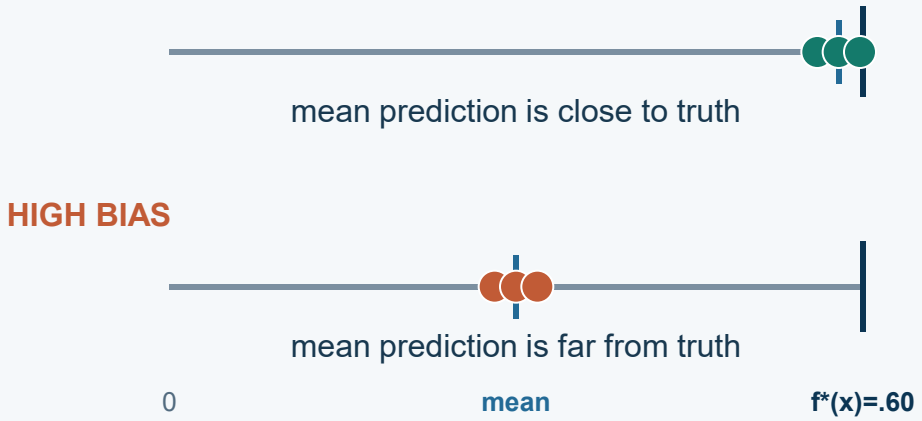
## Standard notation

$f_D(x)$: prediction from the tree trained on D

$\mu(x) = E_D[f_D(x)]$
f*(x): true conditional probability

$\mathbf{Bias(x) = E_D[f_D(x)] - f^*(x)}$
**Distance between the mean prediction and truth**

## Same spread, different distance from truth




**High bias means consistent error; shallow trees often underfit.**

# Variance is instability across fitted trees

Fix an input x, then retrain the same tree-building method on many possible datasets D.

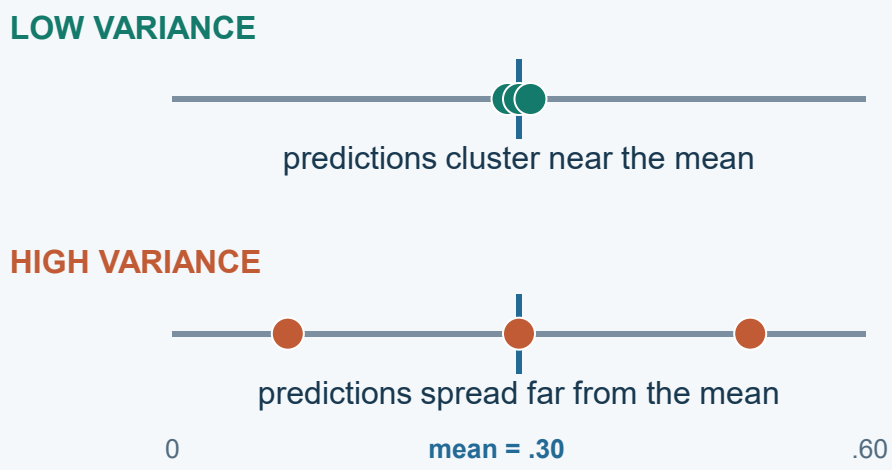
## Standard notation

$f_D(x)$: prediction from the tree trained on D

$\mu(x) = E_D[f_D(x)]$
Mean prediction over possible training sets

$\mathbf{Var_D[f_D(x)] = E_D[(f_D(x) - \mu(x))^2]}$
**Expected squared distance from the mean**

## Same mean, different spread




**High variance means small data changes produce different trees; pruning reduces it.**

# Limit tree complexity before or after growth

Both methods trade a small increase in training error for a simpler tree that often generalizes better.

## Pre-pruning: reject a split first

- Maximum depth
- Minimum leaf size
- Minimum impurity reduction

**A split that creates one-sample leaves fails a minimum leaf size of 2.**

## Post-pruning: grow, then simplify

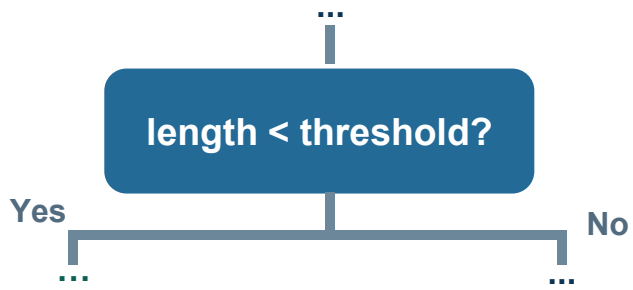
Replace an internal decision node and all its descendants with a single leaf.

Compare the subtree with a single leaf; prune when the simpler option performs as well or better on validation data.

**If the parent predicts “not spam,” prune the two-leaf “sender known?” subtree.**

**Choose growth limits or pruning strength with validation data—not the final test set.**

# How to evaluate impurity reduction for numerical features




**What if we had a numerical email length feature?**
**We can split using a threshold: Length < Threshold?**
**Here, every candidate split leaves some class mixing.**
**Sort the samples, score every midpoint, and choose the largest impurity reduction.**

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Length: | 30 | | 45 | | 70 | | 90 | | 120 | | 160 | | 210 |
| Class: | S | | N | | N | | S | | S | | N | | S |
| Cut: | | ↑ | | ↑ | | ↑ | | ↑ | | ↑ | | ↑ | |
| Threshold: | | 37.5 | | 57.5 | | 80 | | 105 | | 140 | | 185 | |

(S = spam, N = not spam)

## Gini reduction for every cut

37.5 → S--NNSSNS → 0.0612
57.5 → SN--NSSNS → 0.0041
80.0 → SNN--SSNS → **0.0850** (best)
105 → SNNS--SNS → 0.0136
140 → SNNSS--NS → 0.0041
185 → SNNSSN--S → 0.0612

## Gini reduction for the length feature: 0.0850.

**Select the feature with the highest Gini reduction for this node.**

# Missing values

## During learning

To calculate impurity reduction for a feature:
- A training sample may have no value for that feature.
- It cannot be assigned to a child.
- Child class counts are therefore undefined.

How should impurity reduction be calculated?

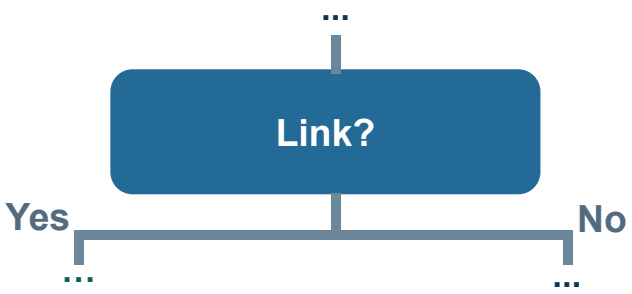
## During prediction

The learned tree asks a question about a feature.
A new sample may have no value for that feature.
- No unique branch is available.
- No leaf prediction is reached.

How should the target be predicted?

**An instance with a missing feature value**

**thread = y, phrase = n, link = ?, att. = y, known = y**



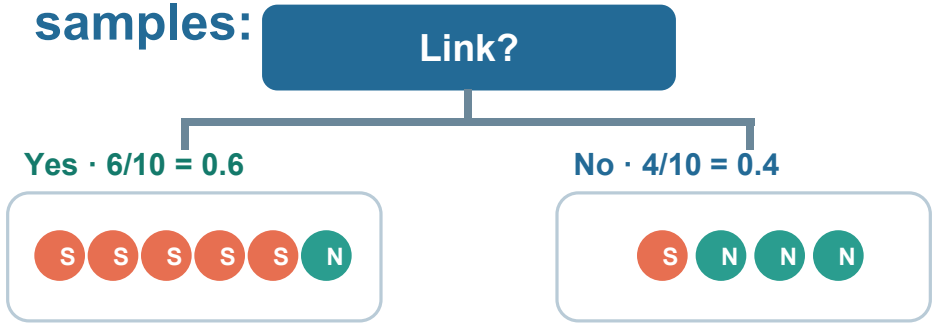
# Fractional routing keeps missing values in both branches

Observed samples determine the branch proportions; those same weights are used during learning and prediction.

## Learning

thread = y, phrase = n, link = ?, att. = y, known = y, class = Spam

### Distribution of the other training samples:




**Split the sample:**

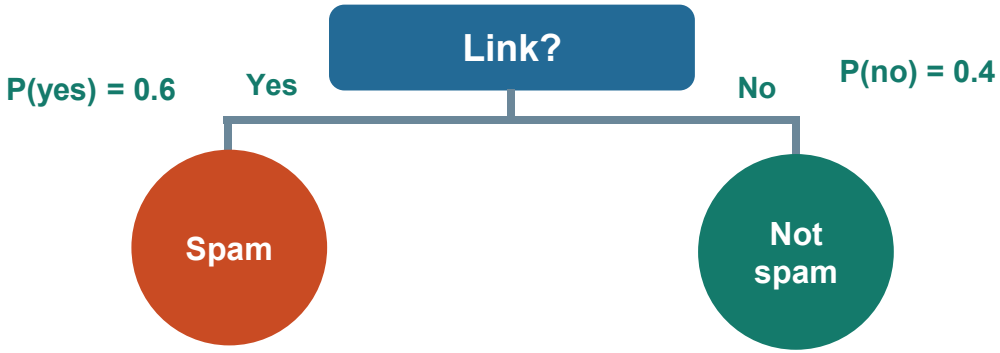
5 S + 0.6 S = 5.6
S
1 N
P(S) = 5.6/6.6
P(N) = 1/6.6

1 S + 0.4 S = 1.4 S
3 N
P(S) = 1.4/4.4
P(N) = 3/4.4

## Prediction

thread = n, phrase = n, link = ?, att. = y, known = n, class = ?

### Distribution of training samples:




P(spam) = 0.6 P(not spam) = 0.4

# Predict numerical targets

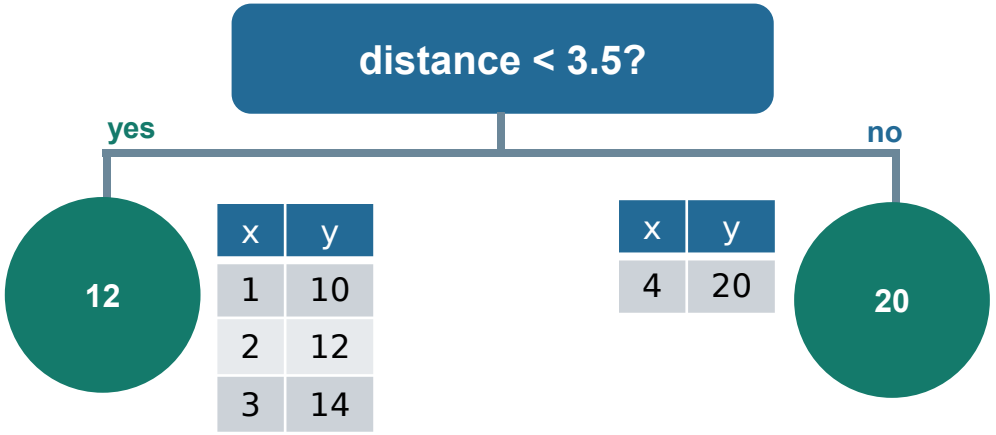
Every leaf predicts the average target of training instances reaching it.

## Example problem

Predict delivery time y from distance x.

| Distance x (km) | Time y (min) |
|---|---|
| 1 | 10 |
| 2 | 12 |
| 3 | 14 |
| 4 | 20 |

**Goal: learn rules that predict time for a new distance.**




Every input reaching the same leaf receives the same numerical prediction.

**new case: x = 2.7, y = ?**

# Learning chooses the split with the largest MSE reduction

For each candidate threshold, fit the mean in each child and compute the weighted child error.

## Candidate thresholds

Parent: y = 10, 12, 14, 20  mean = 14  MSE = 14

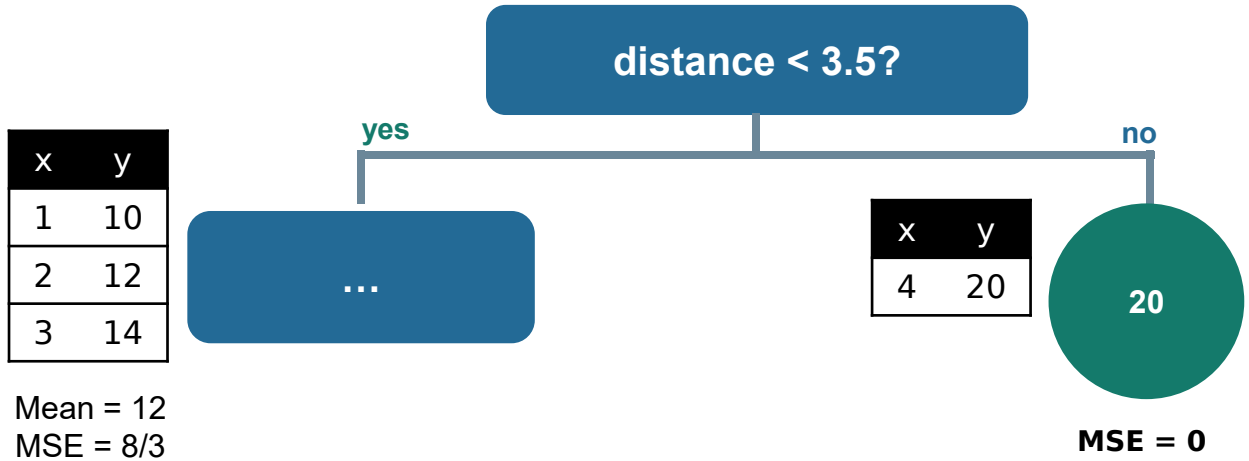
| Cut | Child means | Weighted MSE | Gain |
|---|---|---|---|
| 1.5 | 10 \| 15.33 | 8.67 | 5.33 |
| 2.5 | 11 \| 17 | 5.00 | 9.00 |
| 3.5 | 12 \| 20 | 2.00 | 12.00 |

**Gain = parent MSE − weighted child MSE**

**Weighted child MSE for distance < 3.5 =**
**(3/4)(8/3) + (1/4)(0) = 2**

**Best gain = 14 − 2 = 12**

| Distance x | Time y |
|---|---|
| 1 | 10 |
| 2 | 12 |
| 3 | 14 |
| 4 | 20 |




Mean = 12
MSE = 8/3

**MSE = 0**

**Repeat this search at each new node until a stopping rule is reached.**

# Self-training tasks

| | | |
|---|---|---|
| 1 | **Information Gain Ratio** | Study IGR and explain how it helps. |
| 2 | **Entropy and Gini** | What are the ranges of entropy and Gini for four classes? |
| 3 | **Spam detection project** | Use the spam_detection_dataset from the course page to implement multiple decision trees. Use different maximum depths as stopping criteria, and plot the performance of the resulting models on the training and validation data. |